

Variables & Data Types

A complete study guide covering variables, naming rules, core data types, the `type()` function, user input, and type casting.

Bridge from Chapter 1

In Chapter 1 you installed Python, set up VS Code, and wrote your first program using `print()`. You also learned basic Python syntax and how to add **comments** with the `#` character. Chapter 2 builds directly on those skills. Every code example in this chapter uses `print()` to show output and `#` comments to explain what is happening, so the transition is seamless.

Quick review snippet:

```
# Chapter 1 recap - print() and comments
print("Hello, World!")    # displays text on the screen

# In Chapter 2 we store values in VARIABLES before printing them
name = "Alice"
print(name)               # prints: Alice
```

Section 1: What Are Variables?

1.1 The Memory Reference Concept

A variable is a **named label** that points to a value stored somewhere in your computer's memory (RAM). Think of RAM as a giant row of numbered boxes. When you write:

```
age = 25
```

Python allocates a memory box, places the integer 25 inside it, and makes the label **age** point to that box. You never need to know the actual memory address - Python handles that for you.

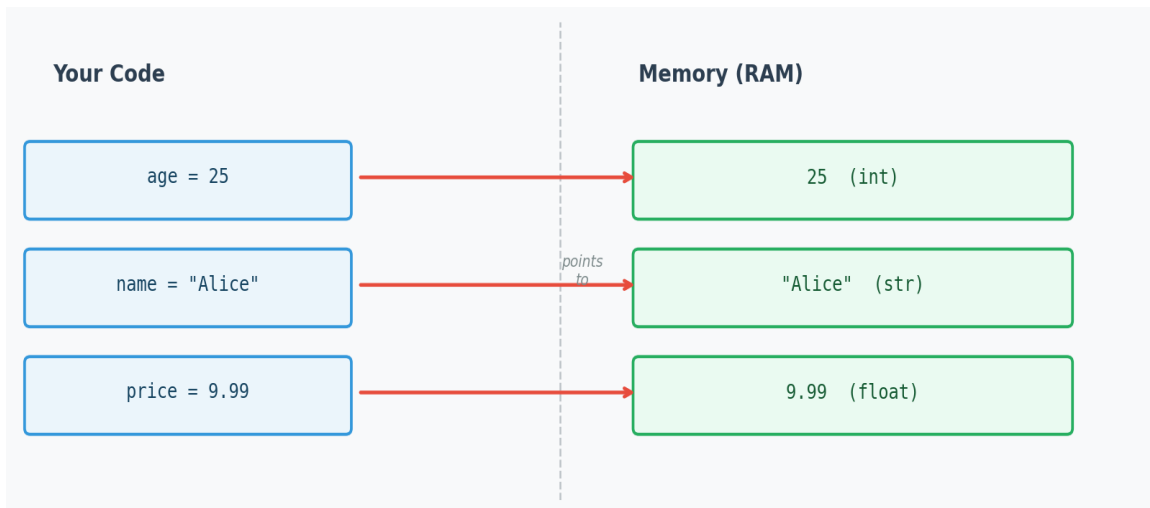


Figure 1: Variables as labels pointing to memory locations.

1.2 Naming Rules and Conventions (PEP 8)

Python has official style guidelines called **PEP 8**. Following them makes your code readable to any Python programmer.

Valid vs. Invalid Variable Names

```
# VALID names
age = 25
first_name = "Bob"           # snake_case: preferred style
total_price = 9.99
user1 = "Alice"              # digits allowed (not at start)
_private = True               # leading underscore is allowed

# INVALID names - these cause SyntaxError
# 1age = 25                   # cannot start with a digit
# first-name = "Bob"          # hyphens are not allowed
# total price = 9.99          # spaces are not allowed
# class = "Math"              # "class" is a reserved keyword
```

PEP 8 Naming Convention: snake_case

Always use lowercase letters with words separated by underscores. This style is called **snake_case** and is the standard in Python.

```
# snake_case (correct Python style)
user_name = "Alice"
total_items = 10
is_logged_in = True

# camelCase (used in other languages, NOT preferred in Python)
```

```
userName = "Alice"          # works, but avoid in Python
```

Reserved Keywords

The following words are **reserved** by Python and cannot be used as variable names. Attempting to do so raises a `SyntaxError`:

```
# Python reserved keywords (partial list)
# if, else, elif, for, while, in, not, and, or,
# True, False, None, def, class, return, import,
# break, continue, pass, try, except, with, as, is

# Example of the error:
# class = "Math"      # SyntaxError: invalid syntax
```

1.3 Multiple Assignment and Constants

Multiple Assignment in One Line

```
# Assign the same value to multiple variables
x = y = z = 0
print(x, y, z)      # 0 0 0

# Assign different values in one line (tuple unpacking)
a, b, c = 10, 20, 30
print(a, b, c)      # 10 20 30

# Swap two variables without a temp variable
first = "hello"
second = "world"
first, second = second, first
print(first, second) # world hello
```

Constants - UPPERCASE Convention

Python does not have true constants (values that cannot change), but the convention is to name constant values using ALL_CAPS with underscores. This signals to other programmers: "do not change this."

```
# Constants by convention (UPPERCASE)
MAX_SPEED = 120
PI = 3.14159
APP_NAME = "MyApp"
DB_HOST = "localhost"

# Python will not stop you from reassigning, but by convention
# you should treat these as fixed values in your program.
print(PI)          # 3.14159
```

```
print(MAX_SPEED) # 120
```

Section 2: Core Data Types

Python automatically determines the data type of a value when you assign it. This is called **dynamic typing** - you do not need to declare the type like you do in languages such as Java or C.

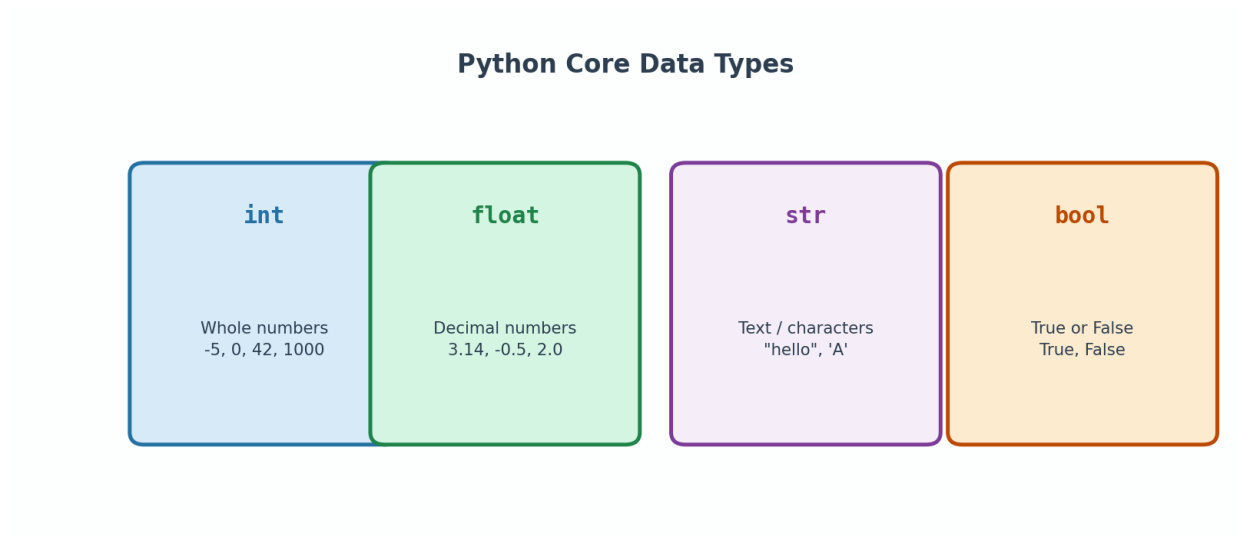


Figure 2: The four core data types covered in this chapter.

2.1 int (Integer)

Integers are whole numbers with no decimal point. They can be positive, negative, or zero. Python supports arbitrarily large integers.

```
# int examples
age = 21
temperature = -5
population = 1_400_000_000    # underscores for readability
zero = 0

print(age)          # 21
print(temperature)  # -5
print(population)   # 1400000000
```

2.2 float (Floating-Point Number)

Floats represent real numbers with a decimal point. They are used for measurements, prices, scientific values, and any calculation that needs precision.

```
# float examples
```

```
price = 9.99
pi = 3.14159
temperature = -0.5
big = 1.5e10          # scientific notation: 15000000000.0
```

```
# Even a whole number becomes a float if you add ".0"
score = 100.0
print(score)          # 100.0
print(type(score))    # <class "float">
```

NOTE: Floating-point arithmetic can produce tiny rounding errors due to how computers represent decimals in binary. For example, 0.1 + 0.2 may equal 0.30000000000000004 instead of 0.3. This is normal behaviour.

2.3 str (String)

Strings are sequences of characters - text. They are enclosed in single quotes, double quotes, or triple quotes for multi-line text.

```
# str examples
name = "Alice"
greeting = 'Hello, World!'
empty = ""          # empty string

# Triple quotes for multi-line strings
message = """Line one
Line two
Line three"""

print(name)         # Alice
print(greeting)     # Hello, World!
print(len(name))    # 5  <- number of characters

# Strings can be joined (concatenated) with +
full_name = "Alice" + " " + "Smith"
print(full_name)    # Alice Smith
```

2.4 bool (Boolean)

Booleans represent one of exactly two values: **True** or **False**. Note the capital first letter - Python is case-sensitive, so "true" or "TRUE" would cause a `NameError`.

```
# bool examples
is_logged_in = True
is_admin = False
has_items = True

print(is_logged_in) # True
```

```
print(is_admin)          # False

# Booleans come from comparisons (Chapter 3 will cover this more)
result = (10 > 5)         # True, because 10 is greater than 5
print(result)            # True

# In Python, True equals 1 and False equals 0 numerically
print(True + True)       # 2
print(False + 5)         # 5
```

2.5 Dynamic Typing

Python infers the data type automatically based on the value you assign. You can even reassign a variable to a completely different type later (though this is generally not recommended for code clarity).

```
# Python figures out the type automatically
x = 10                # Python sees: no decimal -> int
x = 10.5              # Python sees: decimal point -> float
x = "hello"           # Python sees: quotes -> str
x = True              # Python sees: True/False -> bool

# Each reassignment changes both value AND type
counter = 0           # int
counter = counter + 1  # still int: 1
print(counter)        # 1
```

Section 3: The type() Function

The built-in **type()** function tells you exactly what data type Python has assigned to a variable or value. It returns a "class" object which you can print or use in comparisons.

```
# Using type() to check data types
print(type(42))        # <class "int">
print(type(3.14))      # <class "float">
print(type("hello"))   # <class "str">
print(type(True))      # <class "bool">
print(type(None))      # <class "NoneType">

# type() works on variables too
age = 25
name = "Bob"
price = 19.99
active = False

print(type(age))       # <class "int">
```

```
print(type(name))      # <class "str">
print(type(price))     # <class "float">
print(type(active))    # <class "bool">
```

Combining type() with print() - a common pattern

```
# Print variable name, value, and type together
score = 95
print("Value:", score, "| Type:", type(score))
# Output: Value: 95 | Type: <class "int">
```

```
# Inline type check during debugging
user_input = "100"      # comes as a string from input()
print(type(user_input)) # <class "str">  <- NOT int!
```

NOTE: The output "" simply means "this value belongs to Python's int class." For now, treat "class" and "type" as synonymous.

Section 4: Taking User Input with input()

The built-in **input()** function pauses your program and waits for the user to type something and press Enter. It then returns what the user typed.

! CRITICAL RULE: input() ALWAYS returns a string (str), even if the user types a number like 42 or 3.14.

```
# Basic input() usage
name = input("Enter your name: ")
print("Hello,", name)      # greets the user by name

# What the user typed is always a string
age = input("Enter your age: ")    # user types: 25
print(age)                        # prints: 25
print(type(age))                  # prints: <class "str">  <- str, NOT int!
```

Why This Matters - The String Trap

```
# This will NOT add numbers - it will join strings!
num1 = input("Enter a number: ")  # user types: 10
num2 = input("Enter another: ")   # user types: 20
print(num1 + num2)                 # prints: 1020  <- NOT 30!

# The fix: convert to int or float first (see Section 5)
num1 = int(input("Enter a number: "))
num2 = int(input("Enter another: "))
print(num1 + num2)                 # prints: 30  <- correct!
```

The Prompt String Argument

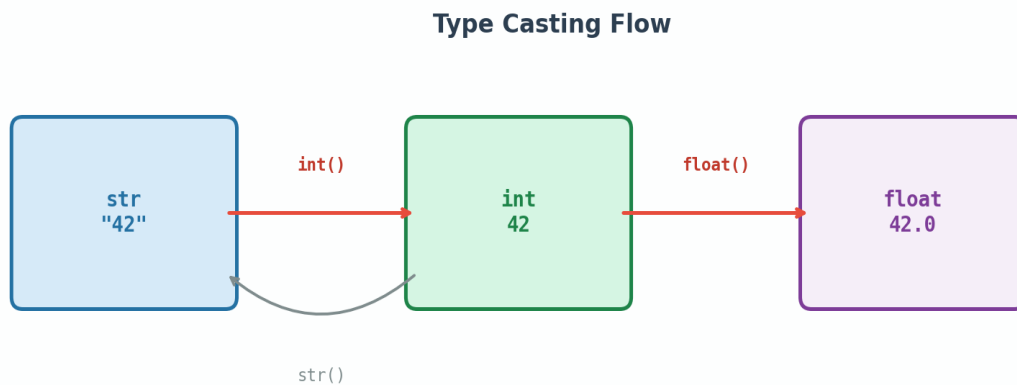
The text you pass to `input()` is shown to the user before they type. It is optional but always include it so users know what to enter.

```
# Good practice: always include a clear prompt
city = input("Enter your city: ")
country = input("Enter your country: ")
print("You live in", city + ", ", country)

# The prompt is just a string - it does NOT affect the returned value
# No prompt (bad practice, user has no idea what to type):
# city = input()
```

Section 5: Type Casting

Type casting (also called type conversion) means converting a value from one data type to another. Python provides three built-in functions for this: `int()`, `float()`, and `str()`.



Note: `int("abc")` raises `ValueError` - only numeric strings can be cast to `int/float`

Figure 3: How type casting converts between data types.

5.1 `int()` - Convert to Integer

```
# str -> int (only works if the string contains a valid whole number)
age_str = "25"
age_int = int(age_str)
print(age_int)           # 25
print(type(age_int))     # <class "int">

# float -> int (truncates the decimal part, does NOT round)
price = 9.99
```



```

price_int = int(price)
print(price_int)          # 9    <- 0.99 is dropped, NOT rounded to 10

# bool -> int
print(int(True))          # 1
print(int(False))         # 0

# Most common real-world use: converting input() to int
age = int(input("Enter your age: "))
print("Next year you will be", age + 1)

```

5.2 float() - Convert to Float

```

# str -> float (works for numeric strings with or without decimals)
price_str = "19.99"
price = float(price_str)
print(price)              # 19.99
print(type(price))        # <class "float">

# int -> float (adds .0)
count = 5
count_f = float(count)
print(count_f)            # 5.0

# "42" is also a valid float string
print(float("42"))        # 42.0

# Real-world use: reading a decimal number from user
weight = float(input("Enter your weight in kg: "))
print("Weight in grams:", weight * 1000)

```

5.3 str() - Convert to String

```

# int -> str
year = 2024
label = "Year: " + str(year)
print(label)              # Year: 2024

# float -> str
pi = 3.14
message = "Pi is approximately " + str(pi)
print(message)            # Pi is approximately 3.14

# bool -> str
print(str(True))          # "True"
print(str(False))         # "False"

```

```
# Important: you CANNOT concatenate str + int directly
# print("Age: " + 25)    # TypeError!
# Fix:
print("Age: " + str(25))  # Age: 25
```

5.4 Common Casting Errors

! Attempting to cast a non-numeric string to int or float raises a ValueError. Handle these carefully.

```
# These will raise ValueError at runtime
# int("abc")      # ValueError: invalid literal for int() with base 10
# int("3.14")     # ValueError: int() cannot convert a float string directly
# float("hello")  # ValueError: could not convert string to float

# CORRECT way to convert "3.14" to int:
# Step 1: convert to float first, then to int
result = int(float("3.14"))
print(result)     # 3

# Beginner tip: always validate input before casting
raw = input("Enter a whole number: ")
if raw.isdigit():
    number = int(raw)
    print("You entered:", number)
else:
    print("That is not a valid whole number!")
```

Section 6: Common Beginner Mistakes

Mistake 1: Using a Variable Before Assigning It

```
# NameError - variable does not exist yet
# print(score)      # NameError: name "score" is not defined

# Fix: always assign before using
score = 0
print(score)        # 0
```

Mistake 2: Case Sensitivity Confusion

```
# Python variable names are CASE-SENSITIVE
Name = "Alice"
name = "Bob"
```

```

NAME = "Carol"

# All three are DIFFERENT variables!
print(Name)    # Alice
print(name)    # Bob
print(NAME)    # Carol

# Common trap: True vs true
# is_valid = true    # NameError: name "true" is not defined
is_valid = True    # correct: capital T

```

Mistake 3: Type Mismatch Errors

```

# Cannot add a string and a number
# result = "Score: " + 100    # TypeError!

# Fix: convert the number to string
result = "Score: " + str(100)
print(result)    # Score: 100

# Forgetting that input() returns a string
age = input("Enter age: ")    # user types 20 -> age = "20" (str)
# print(age + 1)                # TypeError: can only concatenate str to str

# Fix: cast immediately
age = int(input("Enter age: "))
print(age + 1)    # works correctly now

```

Mistake 4: Forgetting str() When Concatenating

```

# Wrong
# print("The answer is " + 42)    # TypeError

# Right option 1: use str()
print("The answer is " + str(42))

# Right option 2: use a comma in print() (auto-adds a space)
print("The answer is", 42)

# Right option 3: use an f-string (covered in later chapters)
print(f"The answer is {42}")

```

Mistake 5: Overwriting a Built-in Name

```

# NEVER use Python built-in names as variable names
# This shadows the built-in and causes confusing errors later

```

```
# Bad:
# input = "hello"      # now input() function is broken!
# type = "int"         # now type() function is broken!
# str = "abc"          # now str() function is broken!

# Good alternatives:
user_input = "hello"
data_type = "int"
text_value = "abc"
```

Chapter 2 Summary and Recap

Key Concepts Covered

- Variables are named labels that point to values in memory.
- Use snake_case for variable names (PEP 8 convention).
- Names cannot start with a digit, contain spaces, or be reserved keywords.
- UPPERCASE names signal constants by convention.
- The 4 core data types: int (whole numbers), float (decimals), str (text), bool (True/False).
- Python is dynamically typed - you do not declare the type.
- type(value) returns the data type of any variable or value.
- input() always returns a str, even when the user types a number.
- int(), float(), str() convert (cast) values between types.
- Casting a non-numeric string to int/float raises a ValueError.

Quick Reference Table

Function	Converts To	Example	Result
int(x)	Integer	int("25")	25
int(x)	Integer	int(3.99)	3
float(x)	Float	float("3.14")	3.14
float(x)	Float	float(5)	5.0
str(x)	String	str(100)	"100"
type(x)	Shows type	type(3.14)	<class "float">
input(x)	String	input("Name: ")	str always

Preview: Chapter 3 - Operators

Now that you understand variables and data types, you are ready for **Chapter 3: Operators**. Operators work directly on the values stored in your variables. For example:

- Arithmetic operators (+, -, *, /, //, %, **) perform math on int and float values.
- Comparison operators (==, !=, <, >, <=, >=) compare values and return a bool.
- Logical operators (and, or, not) combine bool values.
- String operators (+ for join, * for repeat) work on str values.

Every operator you will learn in Chapter 3 requires you to understand the data type it is working on. A solid grasp of Chapter 2 is your foundation for everything that follows.